

Bitácora de pruebas de seguridad web

Objetivo: OWASP Hackademic CMS (playground.kontinuagroup.com) · Retos 1, 2, 5, 6, 7 y 10

SI6007 Desarrollo de Sistemas Seguros · Universidad EAFIT · 09/10/2026

Autor: Sergio Alfredo Junca Valero · Versión web: <https://sjunka.github.io/DesarrolloDeSistemasSeguros/>

Alcance y metodología

Prueba de caja negra autorizada sobre la instancia del curso. Para cada reto: reconocimiento (fuente HTML/JS, recursos, cabeceras, formularios), identificación de la vulnerabilidad, explotación mínima para obtener la bandera, captura de evidencia y propuesta de remediación. Herramientas: navegador con DevTools y Playwright para las capturas.

Resumen de hallazgos

ID	Reto	Vulnerabilidad	OWASP	Severidad
H-01	El agente desaparecido	Datos sensibles en la fuente + listado de directorios	A01 / A05 · Control de acceso y mala configuracion	Alta
H-02	La clave que viaja en el navegador	Validacion de contrasena del lado del cliente	A04 · Diseno inseguro	Alta
H-05	El portero que cree en el User-Agent	Control de acceso basado en cabecera User-Agent	A01 · Perdida de control de acceso	Media
H-06	El codigo ofuscado del caballero	Seguridad por oscuridad (JavaScript ofuscado)	A04 · Diseno inseguro	Alta
H-07	Las notas privadas de la estudiante	Exposicion de informacion + control de acceso roto	A01 / A05 · Control de acceso y mala configuracion	Alta
H-10	El campo oculto que dice 'False'	Confianza en campos ocultos del formulario	A01 · Perdida de control de acceso	Media

Conclusión

Patrón común: **confiar en el cliente**. Credenciales, validaciones, controles de acceso y secretos están en HTML/JS, cabeceras o campos manipulables por el usuario. Todo lo que llega al navegador es público; la autorización y la validación deben ocurrir en el servidor. Prioridad: control de acceso del lado del servidor (A01), retirar secretos del cliente y deshabilitar el listado de directorios (A05).

H-01 · Reto 1: El agente desaparecido

Vulnerabilidad	Datos sensibles en la fuente + listado de directorios
OWASP Top 10	A01 / A05 · Control de acceso y mala configuracion
Severidad estimada	Alta

Contexto

"Our agents informed us that there is reasonable suspicion that the site of this Logistics Company is a blind for a human organs smuggling organisation. Your objective is to find the missing agent's email address." Sitio de la "Greek Logistics Company". Hay que entrar y hallar el correo del agente desaparecido.

Análisis (cómo piensa el atacante)

Primera lección del hacking web: **lo que el navegador recibe, el atacante lo lee todo**. Credenciales en la página, imágenes que apuntan a carpetas y archivos servidos sin control no están protegidos: solo están fuera de la vista. Antes de forzar nada, *leer la fuente y seguir cada recurso* (la pista "white rabbit": seguir al conejo).

Reconocimiento

- Leer el código fuente de la portada (Ctrl+U): las credenciales `white`, `rabbit` están escritas en blanco sobre fondo blanco, invisibles al renderizar.
- Tras entrar, inspeccionar cada imagen: una apunta a una carpeta oculta.
- Probar esa carpeta directamente (listado de directorios activo).

Explotación paso a paso

1. Entrar con las credenciales de la fuente

La portada no muestra credenciales a simple vista: solo el formulario de login. Al abrir el código fuente (Ctrl+U / view-source) aparecen en la línea 30, escritas en **blanco sobre fondo blanco** (`color="#FFFFFF"`): usuario `white`, clave `rabbit`.

*Credenciales en texto plano en el HTML, apenas ocultas con el color del texto. Es **seguridad por oscuridad**: esconderlas a la vista no las protege, porque todo lo que llega al navegador se lee en la fuente.*

usuario: white · clave: rabbit

GREEK LOGISTICS COMPANY
We are the first company in Greece since 1990 that offer human transportation through the internet

Anonymous Corporation since 1990. Reg. No: K7827-232-210B/1990

Enter Code / Password

Enter code and password to enter the transportation system.

Evidencia: Portada de Greek Logistics Company: el login tal como se ve; las credenciales no aparecen en la página renderizada.

view-source:https://playground.kontinuagroup.com/challenges/ch001/index.php

```
1 <html>
2
3 <head>
4 <meta http-equiv="Content-Language" content="en-us">
5 <meta name="generator" content="Bluefish 2.0.2" >
6 <meta name="ProgId" content="FrontPage.Editor.Document">
7 <meta http-equiv="Content-Type" content="text/html; charset=windows-1253">
8 <title>Challenge 001</title>
9 <style type="text/css">
10 .style2 {
11     font-size: xx-large;
12     color: #0000FF;
13 }
14 .style3 {
15     color: #808000;
16 }
17 </style>
18 </head>
19
20 <body >
21     <table border="1" cellpadding="0" cellspacing="0" style="border-collapse: collapse" bordercolor="#111111"
22 width="101%" id="AutoNumber1" height="104">
23     <tr>
24         <td width="85%" height="104">
25             <div style="background-color: #C0C0C0">
26                 <p align="center"><span lang="el" class="style2">GREEK
27 LOGISTICS COMPANY</span><b><font size="7" color="#0000FF"><span
28 lang="el"><br>
29                 </span></font></b><span lang="el"><font size="2" color="#FF0000">We are the first company in Greece since 1990
30 that offer human transportation through the </font></span><font size="2" color="#FF0000"> internet</font></div>
31             </td>
32         </tr>
33     </table>
34
35     <p><i><span class="style3"><span lang="el">Anonymous Corporation since 1990. Reg. No: K7827-232-210B/1990</span></span>
36 <b><font color="#808000"> </font> <font color="#FFFFFF"><span lang="el"> </span>white, rabbit</font></b></i></p>
37
38     <form action="./main/index.php" method="post">
39         <p align="center">Enter Code / Password
40             <input type="text" name="name1" size="20">
41             <input type="text" name="name2" size="20">
42
43         </p>
44
45         <p align="center">
46             <input type="submit" name="submit" value="Enter!">
47         </p>
```

Evidencia: view-source de ch001/index.php (línea 30 resaltada): las credenciales `white`, `rabbit` van con color de texto `#FFFFFF` (blanco) sobre fondo blanco; por eso son invisibles al renderizar.

Enviar el formulario y pasar al área segura (frames con el menú de la empresa).

2. Seguir al conejo hasta la lista de correos

Abrir `mails.php` (Special Clients' Mailbox): tiene una imagen única `./secret_area_/mails.gif`.

Inspeccionando la imagen se ve que vive en la carpeta `secret_area_`. Esa ruta es la pista oculta.

Special Clients' Mailbox

We currently have 5 special clients, who will help us in achieving our goals.

This list will soon grow through the new programme for the expansion of our corporate goals.

It is estimated that we will have reached 20 special customers within the next 6 months.

[Home](#)

Evidencia: Buzón con la imagen que delata la carpeta `secret_area_`

Abrir la carpeta `main/secret_area_`: el servidor (Apache) lista su contenido y expone `mails.txt`.

Listado de directorios activado (mala configuracion): deja ver y descargar archivos no enlazados.

https://playground.kontinuagroup.com/challenges/ch001/main/secret_area_/

Index of /challenges/ch001/main/secret_area_

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 mails.gif	2026-05-09 21:25	71	
 mails.txt	2026-05-09 21:25	655	

Apache/2.4.68 (Debian) Server at playground.kontinuagroup.com Port 80

Evidencia: Listado de directorios de secret_area_: Apache expone mails.gif y mails.txt sin control de acceso.

En mails.txt esta la lista; el agente es el de cumpleaños viernes 13: **Friday13@JasonLives.com** ("Jasson Killer", referencia a Jason / viernes 13).

Respuesta del reto: Friday13@JasonLives.com. El mensaje final se envia desde *Diaxirisths.php* (Central Communication Panel).

```
Everyone is here... xexexe!
-----
Crazy Alice Alice@InWonderland.com
Nebu Chadnezzar NebuChadnezzar@OldKing.edu
Jo Raimontilinekergrobelaar ShortName@badmail.com
Web Killer WebMurder@killer.ever.com
Don Quixote windmill@mail.spain
Crazy priest Exorcist@hotmail.com
Jasson Killer Friday13@JasonLives.com
Everything All AllweSaid@mail.com
Thiseas Sparrow Pirates@mail.gr
Black Dreamer SupaHacka@mail.com
Bond James MyNameIsBond@JamesBond.com
Poor Boy Millionaire@fmail.com
Blind Lynxeyed Linxblind@siou.com
Earl Dracula CarpathianServers@Blood.com
Tea Coffee sugar@dring.com
Whisky Vodka drink@drunk.com
```

Evidencia: mails.txt: el agente desaparecido es Friday13@JasonLives.com

Vías alternativas

- Si el login no cayera por la fuente: **SQL injection** en el formulario con ' OR '1'='1' -- .
- Spidering con Burp Suite / OWASP ZAP para descubrir secret_area_/ y mails.txt sin inspeccionar a mano.

Remediación (desarrollo seguro)

- Nunca poner credenciales en HTML, JS ni comentarios: todo lo que llega al cliente es publico.
- Deshabilitar el listado de directorios (nginx/apache) y no servir archivos de datos como mails.txt.
- Control de acceso real en el servidor para cada recurso; esconder la URL no es control.
- Una lista de correos es dato personal (Ley 1581 en Colombia): no debe quedar accesible.

H-02 · Reto 2: La clave que viaja en el navegador

Vulnerabilidad	Validacion de contraseña del lado del cliente
OWASP Top 10	A04 · Diseño inseguro
Severidad estimada	Alta

Contexto

Sitio militar "TOP SECRET". Pide una contraseña; el botón llama a una función JavaScript `GetPassInfo()` que arma la clave a partir de una cadena. Objetivo: obtener la contraseña.

Análisis (cómo piensa el atacante)

Regla de oro: **nunca confíes en el cliente**. Si la lógica que arma/compara la clave corre en el navegador, el atacante la tiene delante: puede leerla o ejecutarla. La contraseña no está protegida, está *expuesta en el código*.

Reconocimiento

- Ver la fuente: la función `GetPassInfo()` define `var madhouuuuuuseeee = "givesacountinatoap lary"` y arma la variable `Wrong` concatenando caracteres por índice.
- Esa `Wrong` es la clave esperada.

Explotación paso a paso

1. Leer la lógica que arma la clave

Copiar la función `GetPassInfo()` de la fuente.

Concatena `charAt()` de la cadena base para formar la clave. Reconstruyendo los índices da el texto en claro.

```
Reto 2 - funcion GetPassInfo() (fuente real del reto)

function GetPassInfo(){
    var madhouuuuuuseeee = "givesacountinatoap lary"

    var a = madhouuuuuuseeee.charAt(0); var d = madhouuuuuuseeee.charAt(3); var r = madhouuuuuuseeee.charAt(16);
    var b = madhouuuuuuseeee.charAt(1); var e = madhouuuuuuseeee.charAt(4); var j = madhouuuuuuseeee.charAt(9);
    var c = madhouuuuuuseeee.charAt(2); var f = madhouuuuuuseeee.charAt(5); var g = madhouuuuuuseeee.charAt(4);
    var j = madhouuuuuuseeee.charAt(9); var h = madhouuuuuuseeee.charAt(6); var l = madhouuuuuuseeee.charAt(11);
    var g = madhouuuuuuseeee.charAt(4); var i = madhouuuuuuseeee.charAt(7); var x = madhouuuuuuseeee.charAt(21);
    var l = madhouuuuuuseeee.charAt(11); var p = madhouuuuuuseeee.charAt(4); var m = madhouuuuuuseeee.charAt(4);
    var s = madhouuuuuuseeee.charAt(17); var k = madhouuuuuuseeee.charAt(10); var d = madhouuuuuuseeee.charAt(3);
    var t = madhouuuuuuseeee.charAt(18); var n = madhouuuuuuseeee.charAt(12); var e = madhouuuuuuseeee.charAt(4);
    var a = madhouuuuuuseeee.charAt(0); var o = madhouuuuuuseeee.charAt(13); var f = madhouuuuuuseeee.charAt(5);
    var b = madhouuuuuuseeee.charAt(1); var q = madhouuuuuuseeee.charAt(15); var h = madhouuuuuuseeee.charAt(6);
    var c = madhouuuuuuseeee.charAt(2); var h = madhouuuuuuseeee.charAt(6); var i = madhouuuuuuseeee.charAt(7);
    var j = madhouuuuuuseeee.charAt(9); var i = madhouuuuuuseeee.charAt(7); var y = madhouuuuuuseeee.charAt(22);
    var g = madhouuuuuuseeee.charAt(4); var p = madhouuuuuuseeee.charAt(4);
    var l = madhouuuuuuseeee.charAt(11); var k = madhouuuuuuseeee.charAt(10);
    var q = madhouuuuuuseeee.charAt(19); var n = madhouuuuuuseeee.charAt(12);
    var m = madhouuuuuuseeee.charAt(4); var o = madhouuuuuuseeee.charAt(13);

    var p = madhouuuuuuseeee.charAt(4)
    var Wrong = (d+""+j+""+k+""+d+""+x+""+t+""+q+""+e+""+b+""+i+""+l+""+j+""+t+""+k+""+i+""+t+""+g+""+q+""+f+""+y)

    /*if (document.forms[0].Password1.value == Wrong)
        location.href="index.php?Result=" + Wrong;
    */
    var rawQS =
"user_id=2&id=2&class_id=1&token=3116593061834349303492D39832669652D830378353542D33839364652D83065832378366464358353183632";
    if (rawQS) {
        location.href="index.php?" + rawQS + "&Result=" + encodeURIComponent(document.forms[0].Password1.value);
    } else {
        location.href="index.php?Result=" + encodeURIComponent(document.forms[0].Password1.value);
    }
}
```

Evidencia: Fuente real: `GetPassInfo()` y la cadena `givesacountinatoap lary`

2. Extraer la contraseña y entrar

Ejecutar la lógica (consola / debugger) o reconstruir `Wrong`: la clave es **enter a coin to play**.

Evaluando los índices de `charAt` sobre `givesacountinatoap lary` sale "enter a coin to play". También sirve poner un breakpoint y leer `Wrong`.

enter a coin to play

Escribir la clave en el formulario: el reto responde **Congratulations!**.

Congratulations!

Evidencia: Congratulations! tras enviar 'enter a coin to play'

Vías alternativas

- En la consola, redefinir la comparacion para que siempre pase, o leer el parametro `Result` de la URL.
- Interpretar la cadena a mano (los indices `charAt`) sin ejecutar nada.

Remediación (desarrollo seguro)

- Validar credenciales SIEMPRE en el servidor; el cliente solo mejora la experiencia, nunca decide.
- No embeber secretos ni logica de autenticacion en JavaScript entregado al navegador.
- Guardar contraseñas con hash fuerte y salteado (bcrypt/argon2) y compararlas en backend.

H-05 · Reto 5: El portero que cree en el User-Agent

Vulnerabilidad	Control de acceso basado en cabecera User-Agent
OWASP Top 10	A01 · Perdida de control de acceso
Severidad estimada	Media

Contexto

El sitio bloquea el acceso: dice que necesitas comprar *p0wnBrowser* ("solo cuesta 3500 euros"). La decision se toma mirando la cabecera HTTP `User-Agent`. Objetivo: entrar sin comprar nada.

Análisis (cómo piensa el atacante)

El `User-Agent` es una cadena que el cliente elige y falsifica a voluntad. Basar una decision de seguridad en un dato que controla el atacante es un control roto por definicion. Es la misma debilidad del UA que se vio en clase para los clonadores: señal de primera linea, no control de acceso.

Reconocimiento

- Leer el bloqueo: exige el navegador *p0wnBrowser*.
- El servidor solo compara el string del User-Agent enviado por el cliente.

Explotación paso a paso

1. Ver el bloqueo

Entrar con un navegador normal: acceso denegado, pide *p0wnBrowser*.



Evidencia: Bloqueo: 'you must buy p0wnBrowser. It only costs 3500 euros'

2. Falsificar el User-Agent

Cambiar el User-Agent del navegador a p0wnBrowser.

Con una extension "User-Agent Switcher", desde DevTools (Network conditions) o con `curl -A`.

```
curl -A "p0wnBrowser"
```

Recargar con el UA falsificado: acceso concedido, **Congratulations!**.



Evidencia: Congratulations! con User-Agent = p0wnBrowser

Vías alternativas

- Interceptar y editar la cabecera con Burp Suite / OWASP ZAP.
- `curl/wget` con `-A/--user-agent` enviando el UA directo.

Remediación (desarrollo seguro)

- No usar el User-Agent como control de acceso: dato no confiable que el cliente controla.
- Autorizar con sesion autenticada del lado servidor (tokens, roles), no con cabeceras falsificables.
- Si se filtra UA (anti-bot), combinarlo con otras senales (rate limit, fingerprint, reputacion IP), nunca como unica barrera.

H-06 · Reto 6: El codigo ofuscado del caballero

Vulnerabilidad	Seguridad por oscuridad (JavaScript ofuscado)
OWASP Top 10	A04 · Diseno inseguro
Severidad estimada	Alta

Contexto

Sitio "HIDDEN SITE - SECRETS & CODES OF KNIGHT EPOCH". Pide un codigo y su fuente trae JavaScript ofuscado (`document.write(unescape("%3C%68..."))`). Objetivo: desofuscar y hallar el codigo.

Análisis (cómo piensa el atacante)

Ofuscar no es cifrar. El codigo ofuscado se ejecuta en el navegador, asi que el atacante lo **desofusca ejecutandolo**: lo que el navegador entiende, el atacante tambien. Esconder una clave en codigo "raro" da sensacion de seguridad, pero el secreto sigue viajando al cliente. Aqui, ademas, un segundo script tiene la comparacion en texto plano.

Reconocimiento

- Ver la fuente: una gran cadena URL-encoded dentro de `unescape()`.
- Un segundo script define `GetPassInfo()` que compara directamente con el codigo.

Explotación paso a paso

1. Desofuscar / leer la fuente

Copiar la cadena de `unescape(...)` y decodificarla (consola: `console.log(unescape("..."))`), o leer el segundo script.

Decodificar revela el HTML/JS real. El segundo script ya trae la comparacion: `if (PassPhrase.value == 'easyyyyyyy!')`.

[illegible]

Evidencia: Fuente real: JavaScript ofuscado con document.write(unescape(...))

2. Entrar con el codigo

El codigo es **easyyyyyyy!**.

easyyyyyyy!

Escribirlo en el campo: el reto responde **Congratulations!**.

A screenshot of a web page with a black background and yellow text. The text reads: "Congratulations!", "HIDDEN SITE", "SECRETS & CODES OF KNIGHT EPOCH", "SINCE YOU HAVE FOUND THIS SITE MEANS THAT YOU ARE A KNIGHT", "ENTER THE CODE.... IF YOU ARE A REAL ONE", and a form with "Check Code" and "eODe" followed by an input field.

Evidencia: Congratulations! tras enviar easyyyyyyy!

Vías alternativas

- Beautifier/deofuscador de JS (js-beautify) o CyberChef (URL-decode) para dejar el código legible.
- Cambiar `document.write` por `console.log` para ver el decodificado sin que se escriba en la página.

Remediación (desarrollo seguro)

- La ofuscación no protege secretos: todo lo que llega al cliente es recuperable; no es control de seguridad.
- Mantener lógica sensible y secretos en el servidor.
- Minificar/ofuscar sirve para tamaño o propiedad intelectual, nunca para ocultar credenciales.

H-07 · Reto 7: Las notas privadas de la estudiante

Vulnerabilidad	Exposición de información + control de acceso roto
OWASP Top 10	A01 / A05 · Control de acceso y mala configuración
Severidad estimada	Alta

Contexto

Página "Module marks for student": pide un nombre y muestra sus notas. Cualquier nombre que no exista responde `ERRNEOUS IMPORT OF DATA!`. Hay una cookie llamada `not_the_cookie_you_are_looking_for` (sensual que distrae). Objetivo: ver las notas privadas de un estudiante.

Análisis (cómo piensa el atacante)

El reto distrae con una cookie "que no es la que buscas". El fallo real es doble: el servidor **filtra un nombre de usuario válido en un archivo accesible** y luego **entrega datos privados sin autorización** a quien pida ese nombre. No importa quien seas: si conoces el nombre, ves sus notas. Es control de acceso roto, no un tema de cookies.

Reconocimiento

- Probar nombres al azar: siempre `ERRNEOUS IMPORT OF DATA!`. Se necesita un nombre válido.
- La carpeta `index_files/` no lista (403), pero el archivo `index_files/lastlogin.txt` sí es accesible y filtra el último usuario.
- La cookie `not_the_cookie_you_are_looking_for` es un sensual: cambiarla no hace nada.

Explotación paso a paso

1. Conseguir un nombre válido (info disclosure)

Leer `index_files/lastlogin.txt`: filtra la última sesión.

Archivo sensible servido sin control: `"Last Login user: Irene Pretty : Irene at 14/3/2000..."`. El nombre a usar es **Irene**.

`https://playground.kontinuagroup.com/challenges/ch007/index_files/lastlogin.txt`

```
Last Login user:
Irene Pretty : Irene
at 14/3/2000 10:59:00am
```

Evidencia: `lastlogin.txt` filtra el usuario válido: Irene

2. Pedir las notas sin autorización

En el formulario "Module marks for student" escribir `Irene` y enviar.

ACME UNIVERSITY

COMPUTER SCIENCE AND TELECOMMS DEPARTMENT

Module marks for student:

Evidencia: Formulario 'Module marks for student'

El sistema devuelve las notas privadas de Irene Pretty (SUN 666-3-6) sin ninguna comprobacion de permisos.

Broken access control: cualquiera que conozca el nombre ve datos privados (notas, identificador). El flag del reto es haber accedido a estas notas.

ACME UNIVERSITY

COMPUTER SCIENCE AND TELECOMMS DEPARTMENT

Module Marks per Semester for:
[Irene Pretty, SUN: 666-3-6](#)

1st Semester:	
Courses	Grades
Calculus I	E
Digital Logic	FAIL
Linear Algebra	FAIL
Introduction to Programming	E
Introduction to Informatics and Telecomms	FAIL

2nd Semester:	
Courses	Grades
Calculus II	
Physics	
Data Structures	
Object-Oriented Programming	
Electronics and their Applications	

Evidencia: Notas privadas de Irene Pretty expuestas sin autorizacion

Vías alternativas

- Enumerar nombres (los que aparecen en otros retos o diccionario) hasta que uno no de ERRONEOUS.
- Interceptar la peticion con Burp Suite para automatizar la prueba de nombres (fuzzing del parametro name).

Remediación (desarrollo seguro)

- No exponer archivos como `lastlogin.txt`; deshabilitar listado y bloquear rutas de datos.

- Autorizar cada consulta: un estudiante solo ve SUS notas (verificar identidad/rol en el servidor en cada endpoint).
- No confiar en que "nadie sabe el nombre" (seguridad por oscuridad); aplicar control de acceso por objeto (evitar IDOR).
- No sembrar cookies/senales falsas como control; la seguridad no depende de distraer al atacante.

H-10 · Reto 10: El campo oculto que dice 'False'

Vulnerabilidad	Confianza en campos ocultos del formulario
OWASP Top 10	A01 · Perdida de control de acceso
Severidad estimada	Media

Contexto

Sitio "n1nJ4.h4x0r.CreW" (...sh0w m3h y0ur n1nJ4 h4x0r sKiLlz...). El formulario tiene un campo oculto `<input type="hidden" name="LetMeIn" value="False">` que gobierna el acceso. Objetivo: entrar y obtener el serial.

Análisis (cómo piensa el atacante)

Un campo `hidden` esta oculto a la vista, no al control del usuario: viaja en el HTML y se edita antes de enviar. Que el servidor crea el valor `LetMeIn` que manda el cliente es confiar en entrada no confiable. El cliente decide un dato que debería decidir el servidor.

Reconocimiento

- Ver la fuente del formulario: hay un `input type=hidden name=LetMeIn value=False`.
- El servidor decide el acceso segun ese valor.

Explotación paso a paso

1. Alterar el campo oculto

Con el editor inline (Inspeccionar elemento) cambiar `value="False"` por `value="True"`.

Tambien sirve interceptar el *POST* con un proxy y editar el parametro, o enviarlo con `curl -d "LetMeIn=True"`.

```
-> value="True"
```

Reto 10 - campo oculto en el formulario (fuente real)

```
<form method="post" action="">
<input type="hidden" name="LetMeIn" value="False">
<input type="password" name="password">
<input type="submit" name="login" value="Login">
<h3><font color="red">...sh0w m3h y0ur n1nj4 h4x0r sK1Lz...</font></h3>
<hr>
</form>
```

Evidencia: Fuente real del formulario con el campo oculto LetMeIn=False

Enviar el formulario con el campo en `True`.

2. Obtener y decodificar el serial

El reto responde con un `alert` URL-encoded; al decodificarlo aparece el serial.

*Decodificar con `decodeURIComponent(...)` o `CyberChef`. Serial real de esta instancia: **TRVN-67Q2-RU98-546F-H1ZT**.*

```
decodeURIComponent("%53%65%72%69%61%6C...") -> Serial Number: TRVN-67Q2-RU98-546F-H1ZT
```

Reto 10 - serial revelado (alert decodificado, dato real)

LetMeIn cambiado de "False" a "True" -> el reto responde con un `alert` URL-encoded.

Alert crudo:

%53%65%72%69%61%6C%20%45%75%6D%62%65%72%3A%20%54%52%56%45%2D%36%37%51%32%2D%52%55%39%38%2D%35%34%36%46%2D%48%31%5A%54

Decodificado (`decodeURIComponent`):

Serial Number: TRVN-67Q2-RU98-546F-H1ZT

Evidencia: Serial revelado tras poner LetMeIn=True y decodificar el alert

Ingresa el serial en el campo de confirmación para completar el reto.

Vías alternativas

- Interceptar y editar el parametro con Burp Suite / OWASP ZAP en vez del editor inline.
- Enviar el POST a mano: `curl -d "LetMeIn=True&password=x"` saltando el formulario.

Remediación (desarrollo seguro)

- No confiar en campos ocultos ni en ningun dato del formulario para decisiones de acceso: decidir en el servidor.
- El estado sensible (autorizacion, precios, cantidades) vive en la sesion del servidor, no en el HTML del cliente.
- No codificar/ocultar secretos con URL-encoding: codificar no es cifrar.